Notification Service

To maintain file consistency across multiple devices, any modification performed on one client must be communicated to other connected clients.

The notification service is responsible for informing clients whenever:

- A file is added
- A file is modified
- A file is deleted
- A file is shared

This helps minimize synchronization conflicts and ensures users always see the latest version of files.

At a high level, the notification service transfers updates to clients whenever events occur in the system.

Approaches for Notification Service

Two major approaches can be used:

- Long Polling
- WebSocket

1. Long Polling

Dropbox uses long polling for file synchronization notifications.

In long polling:

- The client sends a request to the server.
- The server keeps the connection open until new updates are available or timeout occurs.
- Once updates are available, the server responds immediately.
- The client reconnects again to maintain synchronization.

2. WebSocket

WebSocket provides:

- Persistent connection
- Full duplex communication
- Real-time bidirectional messaging

Both client and server can send messages anytime without reopening connections.

Why Long Polling is Chosen

Although both approaches work, long polling is preferred for Google Drive-like systems for the following reasons:

Reason 1: Communication is Mostly One-Way

The notification service mainly sends updates from server → client.

Clients rarely send real-time messages back to the notification service.

Thus, bidirectional communication provided by WebSocket is unnecessary.

Reason 2: File Notifications are Infrequent

Unlike chat applications, file systems do not continuously generate large bursts of real-time messages.

Notifications occur relatively infrequently, such as:

- File updated
- File uploaded
- File deleted

Long polling is sufficient for such workloads.

Working of Long Polling

Each client establishes a long poll connection with the notification service.

The workflow is as follows:

1. Client sends a long polling request.
2. Notification service keeps the connection open.
3. If file changes occur:
    - Server responds immediately.
    - Client closes the connection.
4. Client requests latest metadata and updates.
5. Client immediately opens a new long polling connection.

This process repeats continuously.

This design ensures:

- Low overhead
- Efficient synchronization
- Real-time-like updates without persistent sockets

Save Storage Space

Supporting file revision history and reliability requires storing multiple copies of files across several data centers.

Without optimization, storage usage can grow extremely fast.

To reduce storage costs, the system uses multiple optimization techniques.

1. De-duplicate Data Blocks

Many files contain identical blocks.

The system removes redundant blocks using block-level deduplication.

Two blocks are considered identical if:

- Their hash values are the same.

Instead of storing duplicate blocks repeatedly:

- Only one copy is stored.
- Multiple files reference the same block.

Benefits:

- Reduced storage usage
- Reduced backup cost
- Improved efficiency

2. Intelligent Backup Strategy

Storing every file revision forever is expensive.

Therefore, the system applies intelligent version management strategies.

Strategy A: Version Limit

The system stores only a fixed number of file versions.

Example:

- Keep only the latest 100 versions.

If the limit is exceeded:

- The oldest version is deleted.

This prevents uncontrolled growth of revision history.

Strategy B: Keep Valuable Versions Only

Some files may be modified hundreds or thousands of times within a short duration.

Saving every revision is wasteful.

Instead:

- Recent versions are prioritized.
- Older insignificant versions may be discarded.

This balances:

- Storage efficiency
- Recovery capability

3. Move Cold Data to Cold Storage

Cold data refers to files that are not accessed for a long period.

Examples:

- Archived documents
- Old backups
- Inactive media files

Such files are moved to cheaper cold storage systems such as:

- Amazon S3 Glacier

Benefits:

- Significant cost reduction
- Long-term archival storage
- Lower storage maintenance cost

Failure Handling

Failures are unavoidable in large-scale distributed systems.

The system must be designed to handle failures gracefully while maintaining:

- Availability
- Reliability
- Data consistency

1. Load Balancer Failure

If a load balancer fails:

- Secondary load balancer becomes active.
- Traffic is redirected automatically.

Load balancers continuously monitor each other using heartbeat signals.

If heartbeat signals stop arriving:

- The load balancer is considered failed.

2. Block Server Failure

If a block server crashes:

- Pending tasks are reassigned to other block servers.
- Unfinished uploads are resumed.

This ensures high availability of upload operations.

3. Cloud Storage Failure

Cloud storage systems like Amazon S3 replicate files across multiple regions.

If one region becomes unavailable:

- Files are fetched from another region.

This guarantees:

- Durability
- Availability
- Disaster recovery

4. API Server Failure

API servers are stateless.

If an API server fails:

- Load balancer redirects traffic to healthy servers.

Since no session state is stored locally:

- Recovery is fast and simple.

5. Metadata Cache Failure

Metadata cache servers are replicated multiple times.

If one cache node fails:

- Requests are served from another replica.
- A new cache node is created to replace the failed server.

This minimizes downtime and maintains fast metadata access.

6. Metadata Database Failure

Database replication ensures high availability.

Master Failure

If the master database fails:

1. One slave is promoted as the new master.
2. A new slave node is created.

This maintains write availability.

Slave Failure

If a slave node fails:

- Read operations are redirected to another slave.
- A replacement slave server is provisioned.

7. Notification Service Failure

Each notification server maintains millions of long poll connections.

According to Dropbox engineering discussions, a single server may maintain over:

- 1 million active connections

If a notification server crashes:

- All long polling connections are lost.
- Clients reconnect to another notification server.

However:

- Re-establishing millions of connections simultaneously is expensive.
- Reconnection must happen gradually.

This makes notification service recovery relatively slow.

8. Offline Backup Queue Failure

Offline backup queues are replicated for fault tolerance.

If one queue fails:

- Consumers re-subscribe to another replicated queue.

This ensures offline clients still receive missed updates when they reconnect.